Atelier Machine Learning

Analyse Comportementale Clientèle Retail

COMPTE RENDU

Atelier Pratique : E-commerce de Cadeaux

Préparé par BOUABIDI RAJA

Année Universitaire : 2025-2026

Table des matières

1 Introduction et Contexte du Projet **2**
1.1 Présentation du contexte métier . . . 2
1.2 Objectifs pédagogiques . . . 2
1.3 Structure du projet . . . 2

2 Configuration de l'Environnement de Travail **3**
2.1 Environnement virtuel . . . 3
2.2 Gestion des dépendances . . . 3

3 Synthèse des Features **3**
3.1 Features numériques (1–34) . . . 3
3.2 Features catégorielles (35–52) . . . 4

4 Problèmes de Qualité des Données **5**

5 Exploration et Préparation des Données **6**
5.1 Standardisation et normalisation . . . 6
5.2 Corrélation et multicolinéarité . . . 6
5.3 Analyse en Composantes Principales (ACP) . . . 6
5.4 Feature Engineering . . . 7
5.5 Imputation des valeurs manquantes . . . 7
5.6 Parsing des données . . . 7

6 Modélisation Avancée **8**
6.1 Séparation Train/Test . . . 8
6.2 Modèles utilisés . . . 8
6.2.1 Clustering (Segmentation) . . . 8
6.2.2 Classification (Churn) . . . 8
6.2.3 Régression (MonetaryTotal) . . . 8
6.3 Optimisation des hyperparamètres . . . 8
6.4 Résultats de la régression . . . 8

7 Déploiement : Interface avec Flask **9**
7.1 Pourquoi Flask ? . . . 9
7.2 Architecture de l'application . . . 9
7.3 Interface utilisateur . . . 9
7.4 Gestion des modèles . . . 10
7.5 Préprocessing à la volée . . . 10

8 Conclusion et Perspectives **10**
8.1 Bilan du projet . . . 10
8.2 Recommandations métier . . . 11
8.3 Perspectives d'amélioration . . . 11

1 Introduction et Contexte du Projet

1.1 Présentation du contexte métier

Ce projet s'inscrit dans le cadre d'un atelier pratique de Machine Learning appliqué à l'analyse comportementale de la clientèle d'une entreprise **e-commerce de cadeaux**. En tant que data scientist, la mission consiste à exploiter une base de données complexe regroupant **52 features** issues de transactions réelles pour :

— **Personnaliser** les stratégies marketing de l'entreprise

— **Réduire** le taux de départ des clients (churn)

— **Optimiser** le chiffre d'affaires par client

Données à disposition

La base de données est intentionnellement imparfaite afin de maîtriser l'ensemble de la chaîne de traitement en data science, de l'exploration jusqu'au déploiement.

1.2 Objectifs pédagogiques

L'atelier vise à développer les compétences suivantes :

Compétence	Description
Exploration	Analyser la qualité et la structure des données
Préparation	Nettoyer, encoder et normaliser les features
Transformation	Réduire la dimension via ACP
Modélisation	Appliquer clustering, classification et régression
Évaluation	Interpréter les résultats et proposer des recommandations
Déploiement	Créer une interface utilisateur avec Flask

Table 1 – Compétences visées par l'atelier

1.3 Structure du projet

L'arborescence du projet respecte les bonnes pratiques de l'ingénierie ML :

```
projet_ml_retail/
|-- data/
|   |-- raw/              # Donnees brutes originales
|   |-- processed/        # Donnees nettoyees et encodees
|   \-- train_test/       # Donnees splittees (X_train, X_test, y_train, y_test)
|-- notebooks/            # Notebooks Jupyter (prototypage)
|-- src/                  # Scripts Python (production)
|   |-- preprocessing.py  # Nettoyage, encodage, normalisation
|   |-- train_model.py    # Entrainement des modeles ML
|   |-- predict.py        # Predictions sur nouvelles donnees
|   \-- utils.py          # Fonctions utilitaires partagees
|-- models/               # Modeles sauvegardes (.pkl, .joblib)
|-- app/                  # Application web Flask
|-- reports/              # Rapports et visualisations
|-- requirements.txt      # Dependances Python
|-- README.md             # Documentation
\-- .gitignore
```

Listing 1 – Structure du projet

2 Configuration de l'Environnement de Travail

2.1 Environnement virtuel

La création d'un environnement virtuel est essentielle pour isoler les dépendances du projet :

```
# Creation de l'environnement virtuel
python -m venv venv

# Activation Windows
venv\Scripts\activate

# Activation Linux/Mac
source venv/bin/activate
```

Listing 2 – Creation et activation de l'environnement virtuel

2.2 Gestion des dépendances

Le fichier `requirements.txt` liste l'ensemble des packages nécessaires. Il se génère automatiquement via `pip freeze` :

```
# Generation du fichier apres installation
pip freeze > requirements.txt

# Reproduction de l'environnement
pip install -r requirements.txt
```

Listing 3 – Gestion des dependances

Package	Version	Usage
numpy	≥1.24.0	Calculs numériques
pandas	≥2.0.0	Manipulation de données
scikit-learn	≥1.3.0	Algorithmes ML
matplotlib	≥3.7.0	Visualisations
seaborn	≥0.12.0	Visualisations statistiques
flask	≥3.0.0	Application web
joblib	≥1.3.0	Sérialisation des modèles
xgboost	≥2.0.0	Gradient boosting
imbalanced-learn	≥0.11.0	Rééquilibrage des classes
optuna	≥3.0.0	Optimisation des hyperparamètres

Table 2 – Principales dépendances du projet

3 Synthèse des Features

La base de données comprend **52 features** réparties en features numériques et catégorielles. Le tableau ci-dessous présente les 34 premières features numériques :

3.1 Features numériques (1–34)

Table 3: Features numériques du dataset

N°	Feature	Type	Description	Unité
1	CustomerID	Entier	Identifiant unique client	–
2	Recency	Entier	Jours écoulés depuis dernier achat	jours
3	Frequency	Entier	Nombre de commandes distinctes	cmd
4	MonetaryTotal	Float	Somme totale dépensée	£
5	MonetaryAvg	Float	Montant moyen par commande	£
6	MonetaryStd	Float	Écart-type des dépenses	£
7	MonetaryMin	Float	Dépense minimale enregistrée	£
8	MonetaryMax	Float	Dépense maximale enregistrée	£
9	TotalQuantity	Entier	Quantité totale d'articles	unités
10	AvgQtyPerTrans	Float	Moyenne articles par commande	unités
11	MinQuantity	Entier	Quantité minimale commandée	unités
12	MaxQuantity	Entier	Quantité maximale commandée	unités
13	CustomerTenure	Entier	Durée relation client	jours
14	FirstPurchase	Entier	Ancienneté premier achat	jours
15	PreferredDay	Entier	Jour semaine favori (0=Lun, 6=Dim)	–
16	PreferredHour	Entier	Heure journée favorite	h
17	PreferredMonth	Entier	Mois année favori	–
18	WeekendRatio	Float	Proportion achats weekend	ratio
19	AvgDaysBetween	Float	Délai moyen entre commandes	jours
20	UniqueProducts	Entier	Nombre produits différents	prod
21	UniqueDesc	Entier	Nombre descriptions uniques	desc
22	AvgProdPerTrans	Float	Produits distincts par cmd	prod
23	UniqueCountries	Entier	Nombre pays associés	pays
24	NegQtyCount	Entier	Lignes quantité négative	occ
25	ZeroPriceCount	Entier	Lignes prix nul	occ
26	CancelledTrans	Entier	Transactions annulées	trans
27	ReturnRatio	Float	Taux retour (qty négatives)	ratio
28	TotalTrans	Entier	Total lignes transactionnelles	lignes
29	UniqueInvoices	Entier	Factures distinctes	fact
30	AvgLinesPerInv	Float	Lignes par facture moyenne	lignes
31	Age	Float	Âge estimé client	ans
32	SupportTickets	Float	Tickets support ouverts	tickets
33	Satisfaction	Float	Note satisfaction	/5
34	Churn	Binaire	Indicateur départ client (0=fidèle, 1=parti)	–

3.2 Features catégorielles (35–52)

Table 4: Features catégorielles du dataset

N°	Feature	Card.	Description	Valeurs	Encodage
35	RFMSegment	4	Segment valeur client	Champions, Fidèles, Potentiels, Dormants	Ord/One-Hot
36	AgeCategory	7	Tranche âge client	18-24, 25-34, ..., 65+, Inconnu	Ordinal
37	SpendingCat	4	Niveau dépense	Low, Medium, High, VIP	Ordinal
38	CustomerType	5	Profil comportemental	Hyperactif, Régulier, Occasionnel, Nouveau, Perdu	One-Hot
39	FavoriteSeason	4	Saison achat préférée	Hiver, Printemps, Été, Automne	One-Hot/Cycl
40	PreferredTime	5	Moment journée favori	Matin, Midi, Après-midi, Soir, Nuit	Ord/One-Hot
41	Region	8	Région géographique	UK, Europe (N/S/E/C), Asie, Autre	One-Hot
42	LoyaltyLevel	5	Niveau ancienneté	Nouveau, Jeune, Établi, Ancien, Inconnu	Ordinal
43	ChurnRisk	4	Niveau risque départ	Faible, Moyen, Élevé, Critique	Ordinal
44	WeekendPref	3	Préférence temporelle	Weekend, Semaine, Inconnu	One-Hot
45	BasketSize	4	Taille habituelle panier	Petit, Moyen, Grand, Inconnu	Ordinal
46	ProdDiversity	3	Diversité achats	Spécialisé, Modéré, Explorateur	One-Hot
47	Gender	3	Genre client	M, F, Unknown	One-Hot
48	AccountStatus	4	État compte	Active, Suspended, Pending, Closed	One-Hot
49	Country	37+	Pays résidence	UK, France, Germany, etc.	Target Enc
50	Newsletter	1	Abonnement newsletter	Yes (unique)	Suppression
51	RegistDate	Var.	Date inscription texte	“12/03/10”, “2010-03-12”	Parsing
52	LastLoginIP	Unique	Adresse IP dernière connexion	“192.168.1.45”	Feature Eng.

4 Problèmes de Qualité des Données

L’analyse exploratoire a révélé plusieurs problèmes de qualité nécessitant un traitement rigoureux :

Type	Features	Fréq.	Traitement
Valeurs manquantes	Age	30%	Imputation (moyenne, médiane, KNN)
Valeurs aberrantes	SupportTickets, Satisfaction	5–8%	Détection et correction
Formats inconsistants	RegistrationDate	100%	Parsing et standardisation
Feature inutile	NewsletterSubscribed	100% cst	Suppression
Données brutes	LastLoginIP	100%	Extraction sous-features
Déséquilibre classes	AccountStatus, Churn	Var.	Rééquilibrage (SMOTE, etc.)

Table 5 – Problèmes de qualité identifiés et traitements appliqués

Point critique : Data Leakage

Il est impératif de ne jamais appliquer `fit_transform` sur l'ensemble du dataset avant le split train/test. Cela constituerait une **fuite de données** (data leakage). De même, la variable cible (y) ne doit **JAMAIS** être normalisée.

5 Exploration et Préparation des Données

5.1 Standardisation et normalisation

Le `StandardScaler` centre les données (moyenne = 0) et réduit (écart-type = 1). Cette étape est essentielle pour les algorithmes sensibles aux échelles :

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)  # Seulement transform !
```

Listing 4 – Application du StandardScaler

5.2 Corrélation et multicolinéarité

La multicolinéarité survient lorsque plusieurs features sont fortement corrélées (ex : `MonetaryTotal` et `MonetaryAvg`). Les solutions appliquées :

- **Matrice de corrélation** avec seuil ($|\rho| > 0{,}8$)
- **VIF** (Variance Inflation Factor) : VIF > 10 = multicolinéarité sévère
- Conservation de la feature ayant le plus de sens métier

5.3 Analyse en Composantes Principales (ACP)

L'ACP permet de réduire la dimension (52 features $\rightarrow$ 2–10 composantes) tout en conservant l'information :

```
from sklearn.decomposition import PCA

pca = PCA(n_components=0.95)  # 95% de variance expliquee
X_pca = pca.fit_transform(X_scaled)
```

Listing 5 – Application de l'ACP

Utilités de l'ACP : visualisation 2D/3D, réduction du bruit, accélération des calculs, évitement de la malédiction de la dimensionnalité.

5.4 Feature Engineering

Création de nouvelles features à partir des existantes :

```
# Ratio depenses / recency
df['MonetaryPerDay'] = df['MonetaryTotal'] / (df['Recency'] + 1)

# Panier moyen
df['AvgBasketValue'] = df['MonetaryTotal'] / df['Frequency']

# Anciennete vs activite recente
df['TenureRatio'] = df['Recency'] / df['CustomerTenure']

# Extraction depuis RegistrationDate
df['RegYear'] = pd.to_datetime(df['RegistrationDate']).dt.year
df['RegMonth'] = pd.to_datetime(df['RegistrationDate']).dt.month
```

Listing 6 – Exemples de feature engineering

5.5 Imputation des valeurs manquantes

Méthode	Usage
Moyenne / Médiane	Données numériques symétriques / asymétriques
Mode	Données catégorielles
KNN Imputer	Moyenne des k plus proches voisins
Iterative Imputer	Régression pour prédire les valeurs manquantes
Valeur constante	-999, "Inconnu" (pour marquer comme manquant)

Table 6 – Méthodes d'imputation utilisées

Important : L'imputation se fait sur `X_train`, puis on applique la **même transformation** sur `X_test`.

5.6 Parsing des données

Exemple de parsing pour la feature `RegistrationDate` aux formats inconsistants :

```
import pandas as pd

# Probleme : formats inconsistants
# "12/03/10" (UK), "2010-03-12" (ISO), "03/12/2010" (US)

# Solution : parser avec to_datetime
df['RegistrationDate'] = pd.to_datetime(
    df['RegistrationDate'],
    dayfirst=True,       # Priorite UK
    errors='coerce'      # NaT si erreur
)

# Extraction de features
df['RegYear'] = df['RegistrationDate'].dt.year
df['RegMonth'] = df['RegistrationDate'].dt.month
df['RegDay'] = df['RegistrationDate'].dt.day
df['RegWeekday'] = df['RegistrationDate'].dt.weekday
```

Listing 7 – Parsing des dates

6 Modélisation Avancée

6.1 Séparation Train/Test

La séparation stratifiée est obligatoire pour préserver la distribution des classes :

```python
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,     # Reproductibilite
    stratify=y           # Conserver proportion classes
)
```

Listing 8 – Separation train/test stratifiee

6.2 Modèles utilisés

Le projet implémente trois types de modélisation :

6.2.1 Clustering (Segmentation)

- **K-Means** : Partitionnement en k clusters
- **DBSCAN** : Clustering basé sur la densité
- **HDBSCAN** : Version hiérarchique de DBSCAN

6.2.2 Classification (Churn)

- **Random Forest** : Forêt d'arbres de décision
- **XGBoost** : Gradient boosting optimisé
- **Régression Logistique** : Modèle linéaire probabiliste

6.2.3 Régression (MonetaryTotal)

- **Ridge** : Régression linéaire régularisée L2
- **Random Forest Regressor** : Forêt pour la régression
- **XGBoost Regressor** : Gradient boosting pour la régression

6.3 Optimisation des hyperparamètres

GridSearchCV vs Optuna

GridSearchCV : Teste toutes les combinaisons possibles d'hyperparamètres via validation croisée. Exhaustif mais coûteux en temps de calcul.
Optuna : Plus intelligent — utilise les résultats précédents pour guider la recherche (plus rapide, moins de combinaisons testées).

6.4 Résultats de la régression

Le graphique ci-dessous présente les performances du modèle Ridge sur la prédiction du `MonetaryTotal` :

Le coefficient de détermination $R^2 = 0{,}8896$ indique que le modèle explique environ **89%** de la variance des données, ce qui constitue une excellente performance pour la prédiction des dépenses futures.

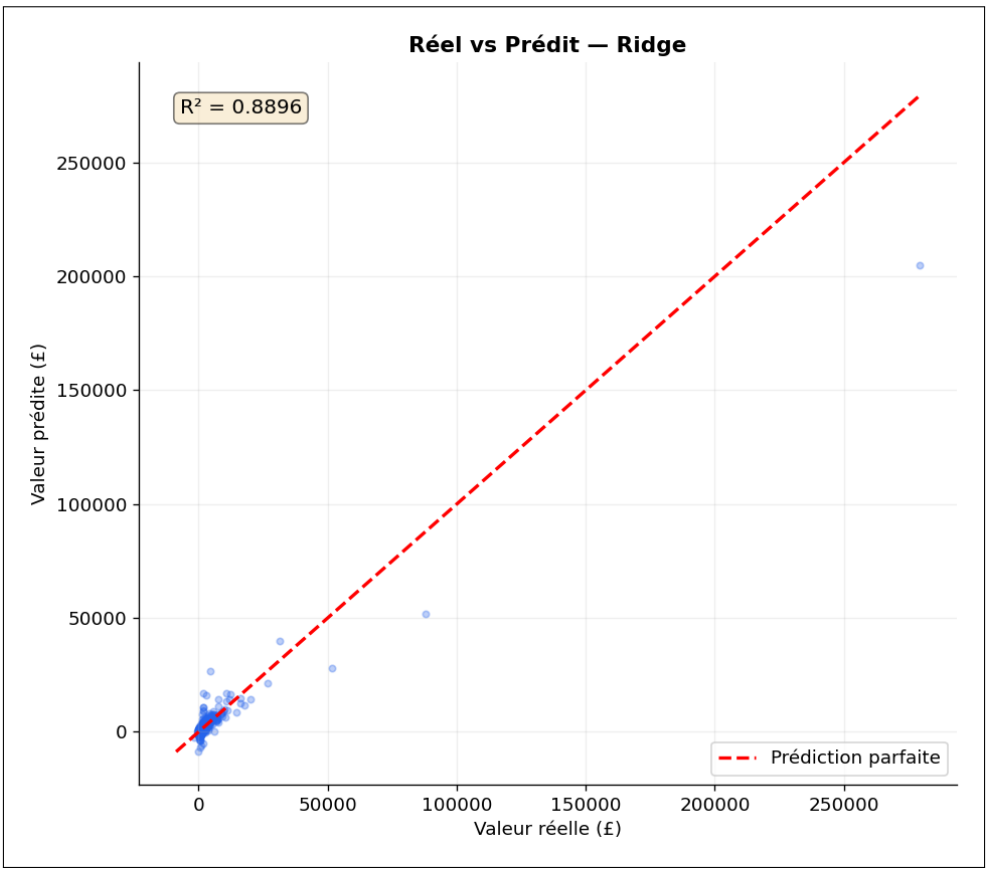

FIGURE 1 – Réel vs Prédit — Modèle Ridge ($R^2 = 0{,}8896$)

7 Déploiement : Interface avec Flask

7.1 Pourquoi Flask ?

Flask est un micro-framework Python léger et flexible, idéal pour :

— Créer des **API REST** pour servir les modèles ML

— Développer rapidement une **interface web** simple

— Intégrer facilement avec **scikit-learn**

— Apprendre les bases du déploiement ML sans complexité excessive

Alternatives : FastAPI (plus rapide, async), Django (plus lourd, complet), Streamlit (très simple pour data apps).

7.2 Architecture de l'application

L'application Flask expose les fonctionnalités suivantes :

7.3 Interface utilisateur

L'interface web comprend :

1. **Dashboard** : Vue d'ensemble avec KPIs (nombre de clients, taux de churn, modèles chargés)
2. **Prédiction individuelle** : Formulaire de saisie des 12 features principales avec résultat détaillé
3. **Batch CSV** : Upload de fichier CSV avec traitement automatique et téléchargement des résultats
4. **Segments** : Visualisation des clusters clients avec métriques par segment

Route	Méthode	Description
`/`	GET	Dashboard avec statistiques
`/predict`	GET/POST	Prédiction individuelle de churn
`/batch`	GET/POST	Prédiction batch via CSV
`/segments`	GET	Visualisation des segments clients
`/api/stats`	GET	API JSON des statistiques
`/health`	GET	Vérification de santé du service
`/download-predictions`	GET	Téléchargement des résultats batch

Table 7 – Routes de l'application Flask

7.4 Gestion des modèles

L'application charge dynamiquement les modèles sauvegardés depuis le dossier `models/` :

```
model_files = {
    "churn_rf":        "models/churn_rf.pkl",
    "churn_lr":        "models/churn_lr.pkl",
    "churn_rf_best":   "models/churn_rf_best.pkl",
    "churn_xgb":       "models/churn_xgb.pkl",
    "cluster_kmeans": "models/cluster_kmeans.pkl",
    "cluster_hdbscan":"models/cluster_hdbscan.pkl",
    "kmeans":          "models/kmeans_model.pkl",
    "pca":             "models/pca_model.pkl",
    "reg_ridge":       "models/reg_ridge.pkl",
    "reg_rf":          "models/reg_rf.pkl",
    "reg_gb":          "models/reg_gb_best.pkl",
    "reg_gb_best":     "models/reg_gb_best.pkl",
}
```

Listing 9 – Chargement des modeles dans app.py

7.5 Préprocessing à la volée

Le système applique automatiquement le préprocessing sur les données brutes en entrée :

```
def preprocess_input(df: pd.DataFrame) -> pd.DataFrame:
    if "Churn" in df.columns:
        df = df.drop("Churn", axis=1)

    if PREPROCESSOR is not None:
        return PREPROCESSOR.transform(df)

    # Fallback : nettoyage minimal
    for col in df.select_dtypes(include=['object']).columns:
        df[col] = pd.to_numeric(df[col], errors='coerce')
    df = df.fillna(df.median(numeric_only=True))
    return df
```

Listing 10 – Fonction de preprocessing

8 Conclusion et Perspectives

8.1 Bilan du projet

Ce projet d'analyse comportementale clientèle a permis de maîtriser l'ensemble de la chaîne de traitement en data science, depuis l'exploration des données jusqu'au déploiement d'une application web fonctionnelle. Les principaux accomplissements sont :

— **Exploration rigoureuse** : Identification et traitement des problèmes de qualité (valeurs manquantes, aberrantes, formats inconsistants)

— **Préparation robuste** : Standardisation, encodage, feature engineering et imputation conformes aux bonnes pratiques (pas de data leakage)

— **Modélisation diversifiée** : Clustering (K-Means, DBSCAN), Classification (Random Forest, XGBoost, Logistic Regression), Régression (Ridge, XGBoost)

— **Évaluation approfondie** : $R^2 = 0{,}8896$ pour la régression Ridge, segmentation en 6 clusters distincts

— **Déploiement opérationnel** : Application Flask avec interface utilisateur complète (dashboard, prédiction individuelle, batch, segments)

8.2 Recommandations métier

Actions prioritaires

1. **Clients à risque critique (≥70%)** : Contacter immédiatement par appel personnalisé, proposer une offre de rétention exclusive
2. **Clients à risque élevé (40–70%)** : Inclure dans la campagne de réengagement mensuelle, activer la séquence email automatisée
3. **Clients fidèles (<40%)** : Maintenir l'expérience actuelle, proposer la montée en gamme vers le programme ambassadeur
4. **Segments VIP et Champions** : Personnaliser les offres premium et le service client dédié

8.3 Perspectives d'amélioration

— Intégration de **FastAPI** pour des performances API supérieures

— Mise en place d'un pipeline **MLflow** pour le tracking des expériences

— Déploiement conteneurisé avec **Docker** et orchestration **Kubernetes**

— Enrichissement des données avec des sources externes (météo, événements, réseaux sociaux)

— Mise en place d'un système de **retraining automatique** des modèles

Projet réalisé dans le cadre de l'Atelier Machine Learning — Année Universitaire 2025-2026